Computer Networks
Assignment 2
Report

Pranay Sanjan Kanneti / IMT2024024

QUESTION 1 : SIMULATING RDT 2.2

CODE IMPLEMENTATION

I have started by making structures for the data packet from sender to receiver in cpp, which includes the sequence number, the actual data (which in this case is a string), and checksum (which is an integer in my simulation which adds up the sequence number and the ASCII values of characters in data).

The structure of acknowledgement packet from receiver to sender contains two fields: the ack number (which in this case will be 0 or 1 as RDT 2.2 is a stop and wait protocol), and the checksum which is same as ack number.

Then I created functions to calculate checksum for data packet and acknowledgement packet, functions to randomly corrupt packets which just randomly change the value in checksum field upon being called, functions to check if packet received (either data packet or acknowledgement packet) is corrupted or not, and finally I created the unreliable channel functions to simulate unreliable channel below the transport layer which calls the `corrupt_randomly` function for random packets that are to be transferred through this unreliable channel.

And now for simulating the processes happening on sender and receiver sides, by following the FSM's of sender and receiver for RDT 2.2:

For the receiver, I started by calculating if the received packet is corrupted or not and if it is in sequence or not. If it is a valid packet, I toggled the expected sequence number and sent the ack packet for just received packet through the unreliable channel. And if the received data packet is not valid, then in the ack packet the ack number will be of the previously correctly received packet, found the checksum and sent the packet. (This receiver function returns the final packet which is delivered at the sender side)

For the sender implementation I set the initial `seq_number` and expected `seq_number` at receiver to 0. As mentioned earlier that my data is in the format of string, in the main function I created an array of strings that are to be transferred to the receiver. Now for each data, I created a packet, gave it a sequence number, calculated the checksum, and in an infinite loop which breaks only when the sender receives a valid ACK, I sent this packet through the unreliable channel. Now I compared the ack number with sequence

number, checked if the ack packet is corrupted or not and if it is valid ACK, I updated the next sequence number and broke out of the loop. If the ack packet is not valid then I try to send the same packet again until a valid ack is received.

PARAMETER CHOICES

In the main function I just set up random number generation which is used in the code, explicitly declared a set of data to be transferred (considering each string as data in that segment). I initialised a float value (0 - 1) named `corruption_probability` (I set its value to 0.3, which can be changed. . .) which controls the extent of packet corruption in the channel.

NOTE

- There is no need to give any input to the code.

QUESTION 2 : SIMULATING RDT 3.0

CODE IMPLEMENTATION

Everything regarding the structures of packets and calculating checksums, verifying checksums is same as of that in RDT 2.2 code.

For simulating the timeout and retransmission logic in RDT 3.0, I created a timer using a thread, mutex, and condition variable. The timer is used to track timeouts if no valid ACK is received for a sent packet. The unreliable channel is simulated using the `transmit` function, where I introduced both packet loss and corruption by checking random probability values.

The receiver function checks if the packet is corrupted using the checksum, and also checks if the sequence number is expected. If valid, the data is accepted, printed, and ACK with the correct sequence number is sent. If the packet is corrupted or out-of-order, it discards the packet and sends ACK for the last correctly received packet.

On the sender side, I used a loop to send each string message as a packet. For each packet, I calculate the checksum and send it through the unreliable channel. I then wait for the ACK. If the received ACK is valid and matches the expected sequence number, I stop the timer, flip the sequence number, and move to the next message. If the ACK is not valid or timeout happens, the same packet is sent again. The sender tries for a maximum number of attempts (20 in this case) before giving up.

```
[Sender] Sending packet: "Hello", seq: 0
[Receiver] Received valid packet: Hello
[Sender] Received ACK: 0, valid ACK. Proceeding to next message.

[Sender] Sending packet: "This is", seq: 1
[Receiver] Received valid packet: This is
[Sender] Received ACK: 1, valid ACK. Proceeding to next message.

[Sender] Sending packet: "the simulation", seq: 0
[Receiver] Corrupted or duplicate packet. Sending duplicate ACK.
[Sender] Received ACK: 1, duplicate ACK. Retransmitting...

[Sender] Sending packet: "the simulation", seq: 0
[Receiver] Corrupted or duplicate packet. Sending duplicate ACK.
[Sender] Received ACK: 1, but it's corrupted. Retransmitting...

[Sender] Sending packet: "the simulation", seq: 0
[Receiver] Received valid packet: the simulation
[Sender] Received ACK: 0, valid ACK. Proceeding to next message.

[Sender] Sending packet: "of rdt2.2", seq: 1
[Receiver] Received valid packet: of rdt2.2
[Sender] Received ACK: 1, valid ACK. Proceeding to next message.

[Sender] Sending packet: "protocol", seq: 0
[Receiver] Corrupted or duplicate packet. Sending duplicate ACK.
[Sender] Received ACK: 1, duplicate ACK. Retransmitting...

[Sender] Sending packet: "protocol", seq: 0
[Receiver] Received valid packet: protocol
[Sender] Received ACK: 0, valid ACK. Proceeding to next message.

[Sender] Sending packet: "done.", seq: 1
[Receiver] Received valid packet: done.
[Sender] Received ACK: 1, but it's corrupted. Retransmitting...

[Sender] Sending packet: "done.", seq: 1
[Receiver] Corrupted or duplicate packet. Sending duplicate ACK.
[Sender] Received ACK: 1, valid ACK. Proceeding to next message.

 All messages sent successfully.
```

Figure 1: Output for the RDT2.2 simulation

PARAMETER CHOICES

In the main function, I initialized a vector of strings as the messages to be sent. The corruption and loss probabilities are both set to 0.3 (which can be changed). These values decide how often packets or ACKs get corrupted or lost during transmission. The timer is set to 1000 milliseconds for each packet, and the sender tries to send each message up to 20 times before stopping if all attempts fail.

NOTE

- No user input is needed to run the code.
- The simulation continues until all messages are successfully transmitted or retries are

exhausted.

- As the image is too big to fit here, you can check the output format in the images attached from the IMAGES folder
- once in 10 times due to the complexity involving regarding the randomness and time from threads if the output does not reach its end then please try running the code again.

QUESTION 3 : SIMULATING TCP TAHOE

NOTE

- To make use of MSS I took the data to be transferred as a stream of characters and this MSS specifies how many characters each packet contains.
- I assumed that the packet loss has occurred at random times, and did not care about retransmitting the lost packets and just continued with transferring the remaining data with new `cwnd` size and new `ssthresh`, as the main moto of the question is to simulate TCP Tahoe.
- FYI: if `cwnd = 3` and `MSS = 5` the sender transfers back to back 3 packets which contain 5 characters each.
- Refer figure 2 for the output image.

IMPLEMENTATION DETAILS

The user must specify the initial value of `cwnd`, `ssthresh`, and `MSS`, in the `tahoe` function which is to transfer the packets made out of the stream of characters, which are to be transferred in pipelined fashion. The data is divided into chunks of size `MSS`, and in each round, multiple chunks are sent based on the current `cwnd`.

If any loss is detected in a round, TCP Tahoe simulates a timeout: `cwnd` is reset to 1 and `ssthresh` is updated to half of the previous `cwnd`. Otherwise, it follows exponential growth in the slow start phase and switches to linear growth once past `ssthresh`, which is the congestion avoidance phase.

QUESTION 4 : SIMULATING TCP RENO

NOTE

- The implementation is mostly same as that of TCP Tahoe, I just added that there could be 2 types of packet losses and hence modified the code accordingly to change

```
Enter the initial value of cwnd: 1
Enter the initial value of ssthresh: 8
Enter the value of MSS_size: 5

Sending chunk: Hello
All packets in round sent. cwnd =  2, ssthresh = 8

Sending chunk: .this
Sending chunk: .is.a
All packets in round sent. cwnd =  4, ssthresh = 8

Sending chunk: .test
Sending chunk: .mess
Sending chunk: age.t
Sending chunk: o.be.
Packet loss detected. new cwnd = 1, new ssthresh = 2

Sending chunk: sent.
All packets in round sent. cwnd =  2, ssthresh = 2

Sending chunk: over.
Sending chunk: TCP.T
Packet loss detected. new cwnd = 1, new ssthresh = 1

Sending chunk: ahoe.
Packet loss detected. new cwnd = 1, new ssthresh = 0

Sending chunk: proto
Packet loss detected. new cwnd = 1, new ssthresh = 0

Sending chunk: col.I
All packets in round sent. cwnd =  2, ssthresh = 0

Sending chunk: .am.u
Sending chunk: sing.
All packets in round sent. cwnd =  3, ssthresh = 0

Sending chunk: this.
Sending chunk: to.te
Sending chunk: st.th
All packets in round sent. cwnd =  4, ssthresh = 0

Sending chunk: e.tah
Sending chunk: oe.pr
Sending chunk: otoco
Sending chunk: l.
Packet loss detected. new cwnd = 1, new ssthresh = 2

All data sent.
```

Figure 2: Output for the TCP Tahoe simulation

the cwnd size to 1 and ssthresh to cwnd(old)/2 if the congestion is due to timeout and the cwnd size to cwnd(old)/2 and ssthresh to cwnd(old)/2 if congestion is due to detection of triple duplicate ack(I have coded such that the probability of detecting triple duplicate ack is more that a packet loss occuring due to timeout but we can change this if needed).

- Refer figure 2 and figure 3 for the output image.
- I recommend the user to take `MSS` as something between $2 - 5$ for better simulation.

```
Enter the value of cwnd: 1
Enter the value of initial ssthresh: 8
Enter the value of MSS_size: 3

Sending chunk: Hel
All packets in round sent. cwnd = 2, ssthresh = 8

Sending chunk: lo.
Sending chunk: thi
All packets in round sent. cwnd = 4, ssthresh = 8

Sending chunk: s.i
Sending chunk: s.a
Sending chunk: .te
Sending chunk: st.
All packets in round sent. cwnd = 8, ssthresh = 8

Sending chunk: mes
Sending chunk: sag
Sending chunk: e.t
Sending chunk: o.b
Sending chunk: e.s
Sending chunk: ent
Sending chunk: .ov
Sending chunk: er.
All packets in round sent. cwnd = 9, ssthresh = 8

Sending chunk: TCP
Sending chunk: .Re
Sending chunk: no.
Sending chunk: pro
Sending chunk: toc
Sending chunk: ol.
Sending chunk: I.a
Sending chunk: m.u
Sending chunk: sin
All packets in round sent. cwnd = 10, ssthresh = 8

Sending chunk: g.t
Sending chunk: his
Sending chunk: .to
Sending chunk: .te
Sending chunk: st.
Sending chunk: the
Sending chunk: .ta
Sending chunk: hoe
Sending chunk: .pr
Sending chunk: oto
Packet loss detected due to recognition of a triple duplicate ACK. new cwnd = 5, new ssthresh = 5

Sending chunk: col
Sending chunk: .
All packets in round sent. cwnd = 6, ssthresh = 5

All data sent.
```

Figure 3: Output for the TCP Reno simulation(1)

```
Enter the value of cwnd: 1
Enter the value of initial ssthresh: 8
Enter the value of MSS_size: 3

Sending chunk: Hel
Packet loss detected due to a packet Timeout. new cwnd = 1, new ssthresh = 0

Sending chunk: lo.
Packet loss detected due to a packet Timeout. new cwnd = 1, new ssthresh = 0

Sending chunk: thi
All packets in round sent. cwnd = 2, ssthresh = 0

Sending chunk: s.i
Sending chunk: s.a
All packets in round sent. cwnd = 3, ssthresh = 0

Sending chunk: .te
Sending chunk: st.
Sending chunk: mes
All packets in round sent. cwnd = 4, ssthresh = 0

Sending chunk: sag
Sending chunk: e.t
Sending chunk: o.b
Sending chunk: e.s
Packet loss detected due to a packet Timeout. new cwnd = 1, new ssthresh = 2

Sending chunk: ent
Packet loss detected due to recognition of a triple duplicate ACK. new cwnd = 0, new ssthresh = 0

All packets in round sent. cwnd = 1, ssthresh = 0

Sending chunk: .ov
Packet loss detected due to recognition of a triple duplicate ACK. new cwnd = 0, new ssthresh = 0

All packets in round sent. cwnd = 1, ssthresh = 0

Sending chunk: er.
All packets in round sent. cwnd = 2, ssthresh = 0

Sending chunk: TCP
Sending chunk: .Re
Packet loss detected due to a packet Timeout. new cwnd = 1, new ssthresh = 1

Sending chunk: no.
Packet loss detected due to recognition of a triple duplicate ACK. new cwnd = 0, new ssthresh = 0

All packets in round sent. cwnd = 1, ssthresh = 0

Sending chunk: pro
Packet loss detected due to recognition of a triple duplicate ACK. new cwnd = 0, new ssthresh = 0

All packets in round sent. cwnd = 1, ssthresh = 0

Sending chunk: toc
All packets in round sent. cwnd = 2, ssthresh = 0

Sending chunk: ol.
Sending chunk: I.a
All packets in round sent. cwnd = 3, ssthresh = 0

Sending chunk: m.u
Sending chunk: sin
Sending chunk: g.t
Packet loss detected due to recognition of a triple duplicate ACK. new cwnd = 1, new ssthresh = 1

Sending chunk: his
All packets in round sent. cwnd = 2, ssthresh = 1

Sending chunk: .to
Sending chunk: .te
Packet loss detected due to a packet Timeout. new cwnd = 1, new ssthresh = 1

Sending chunk: st.
All packets in round sent. cwnd = 2, ssthresh = 1

Sending chunk: the
Sending chunk: .ta
All packets in round sent. cwnd = 3, ssthresh = 1

Sending chunk: hoe
Sending chunk: .pr
Sending chunk: oto
All packets in round sent. cwnd = 4, ssthresh = 1

Sending chunk: col
Sending chunk: .
All packets in round sent. cwnd = 5, ssthresh = 1

All data sent.
```

Figure 4: Output for the TCP Reno simulation(2)